

Contract Testing

Consumer ↔ Provider

Kiểm chứng khả năng giao tiếp giữa các dịch vụ — nhanh, cô lập và đủ sớm để bảo vệ CI/CD.

Consumer-Driven

Pact workflow

Architecture

Khi mỗi service đều “**xanh**”... hệ thống vẫn có thể “**đỏ**”

Consumer

Giả định sai

Client kỳ vọng `product.name`, nhưng
Provider đổi thành `displayName`.

Provider

Test nội bộ vẫn pass

Logic và schema mới đều đúng theo
góc nhìn của đội backend.

Runtime

Lỗi chỉ lộ khi tích hợp

Phát hiện muộn trên staging hoặc
production, nơi feedback đắt đỏ nhất.

Khoảng trống cần kiểm thử: “Hai phía có còn hiểu cùng một giao thức hay không?”

Contract Testing kiểm tra **lời hứa giao tiếp**

Contract là đặc tả có thể thực thi về những request Consumer gửi và response Provider cam kết đáp ứng.

- **Request:** method, path, query, headers, body
- **Response:** status, headers, schema và matching rules
- **Context:** provider state — điều kiện trước của interaction

INTERACTION

Given product 10 exists

When GET /products/10

Then 200 + Product schema

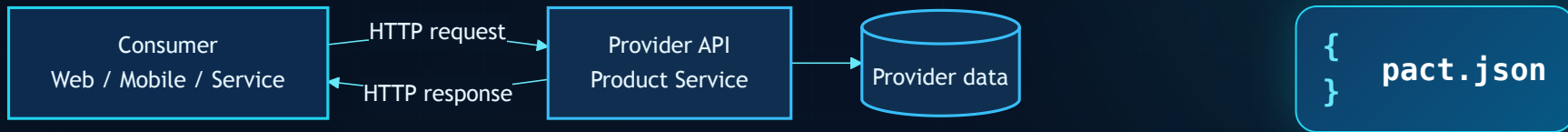
Contract Testing xác minh **tính tương thích** — không chứng minh toàn bộ nghiệp vụ đúng.

Contract Test nằm ở đâu trong chiến lược kiểm thử?

Tiêu chí	Integration	Contract	End-to-End
Câu hỏi chính	Các module phối hợp đúng?	Giao diện có tương thích?	Hành trình người dùng chạy được?
Phạm vi	Một nhóm thành phần	Một cặp Consumer–Provider	Toàn hệ thống
Môi trường	Bán tích hợp	Cô lập, local/CI	Gần production
Feedback	Trung bình	Nhanh, định vị rõ	Chậm, khó khoanh vùng
Điểm mù	Phụ thuộc ngoài scope	Business flow, hạ tầng	Nhiều nguyên nhân gây flaky

Contract Test **bổ sung** Unit / Integration / E2E — không thay thế tất cả.

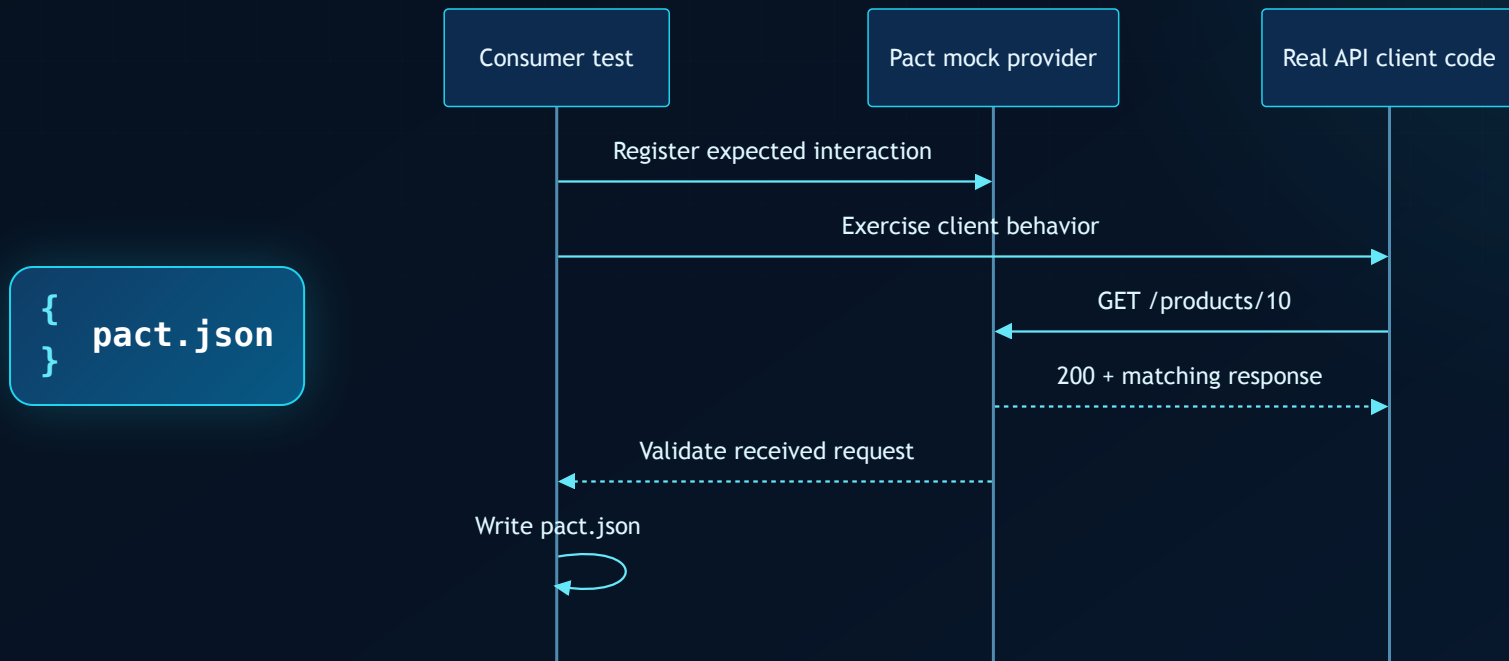
Ai sở hữu **lời hứa** nào?



Consumer mô tả phần API nó thực sự sử dụng.

Provider chứng minh implementation đáp ứng các interaction đó.

Bước 1 — Consumer **tạo contract**



Mock Provider không thay thế code của Consumer: test phải gọi **API client thật** của Consumer.

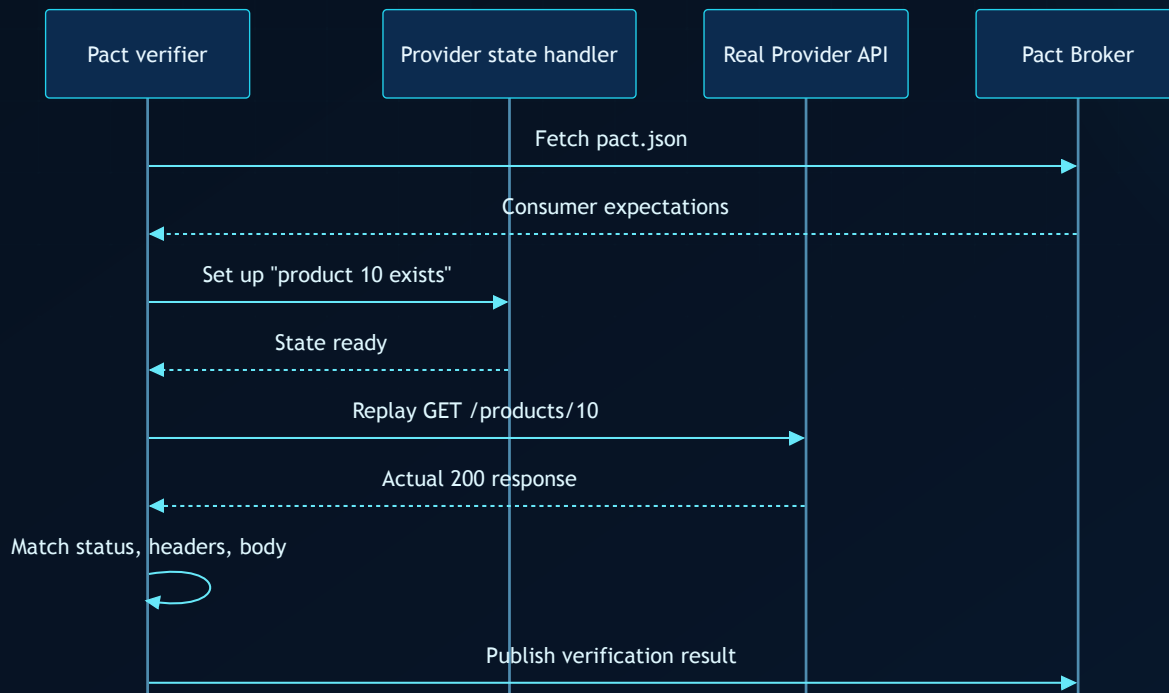
Một interaction tốt = ví dụ cụ thể + quy tắc linh hoạt

```
{
  "description": "get product 10",
  "providerState": "product 10 exists",
  "request": {
    "method": "GET",
    "path": "/products/10"
  },
  "response": {
    "status": 200,
    "body": {
      "id": "10",
      "name": "28 Degrees",
      "type": "CREDIT_CARD"
    }
  },
  "matchingRules": {
    "$.body.id": "type:string",
    "$.body.name": "type:string"
  }
}
```

Nguyên tắc:

strict với điều Consumer phụ thuộc;
linh hoạt với dữ liệu động.

Bước 2 — Provider **xác minh contract**

`{ } pact.json`**Real routing**

Request đi vào API thật.

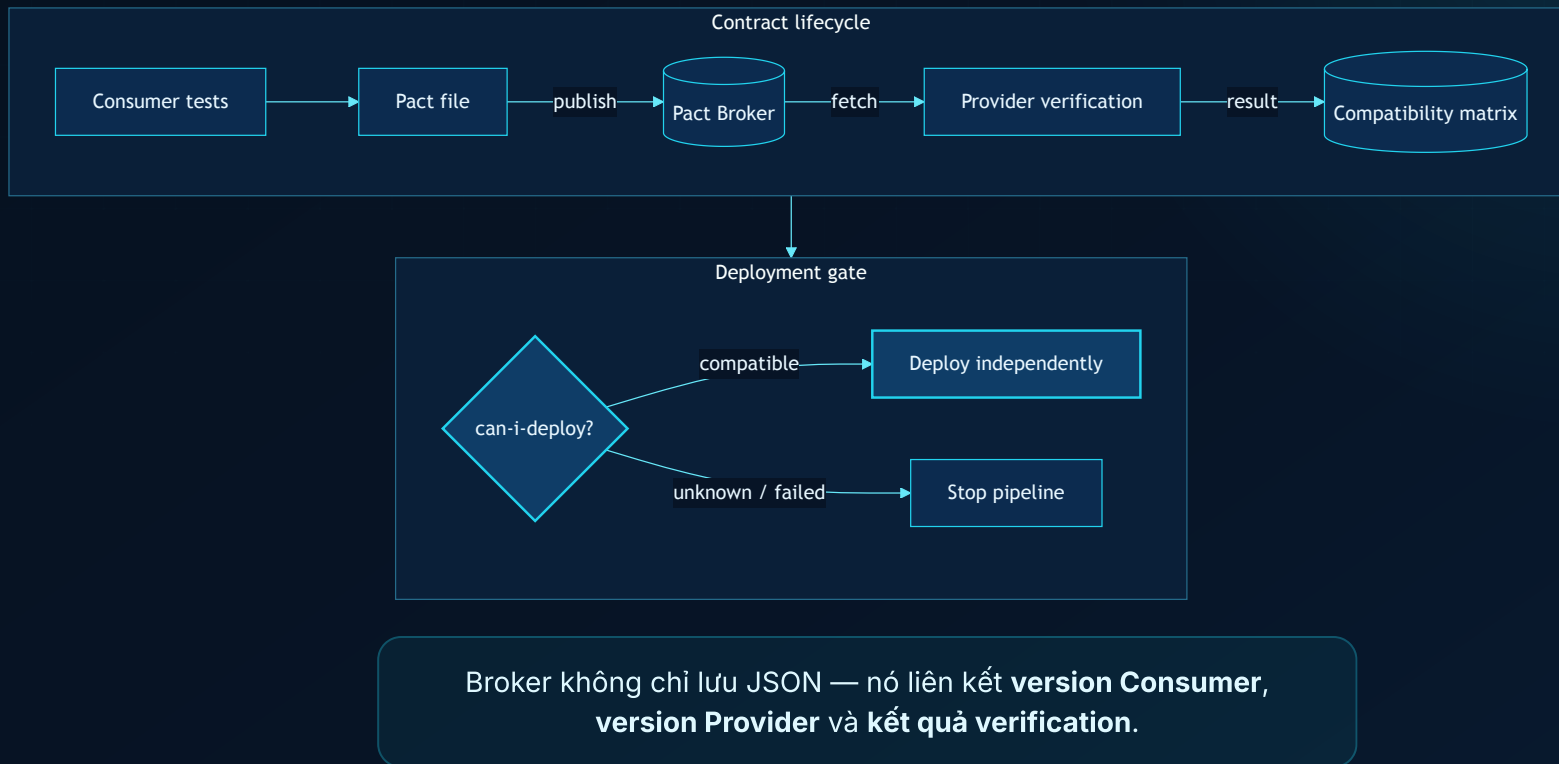
Controlled state

Dữ liệu test có thể tái tạo.

Precise diff

Mismatch chỉ rõ field bị gãy.

Broker biến contract thành điểm đồng bộ



Vì sao gọi là **Consumer-Driven**?

Provider-Driven

Provider công bố toàn bộ schema/API. Consumer phải thích nghi, nhưng Provider khó biết phần nào đang thực sự được sử dụng.

Consumer-Driven

Mỗi Consumer phát hành các interaction mình phụ thuộc. Provider xác minh tập hợp nhu cầu thực tế đó.

1 Consumer nêu nhu cầu

2 Provider xác minh

3 Hai đội cùng tiến hóa API

Công cụ hỗ trợ cuộc hội thoại — không thay thế thiết kế API và giao tiếp giữa các đội.

Contract tốt kiểm tra **hành vi**, không đóng băng dữ liệu

Interaction có ý nghĩa

Đặt tên theo hành vi: "get existing product", không theo implementation nội bộ.

Matcher vừa đủ

Dùng type/regex cho dữ liệu động; giữ exact match cho status và giá trị nghiệp vụ quan trọng.

State độc lập

Mỗi interaction tự thiết lập provider state; không phụ thuộc thứ tự chạy.

Ít hơn nhưng sắc

Chỉ contract các field Consumer sử dụng. Tránh sao chép toàn bộ response "cho chắc".

Version minh bạch

Gắn pact và verification với commit/branch để ma trận tương thích có ý nghĩa.

Chạy trong CI

Publish, verify và deployment gate phải tự động — không dựa vào kiểm tra thủ công.

Contract Testing **không nhìn thấy** điều gì?

Business logic nội bộ

Response đúng schema nhưng tính sai tổng tiền vẫn có thể pass.

Hạ tầng production

DNS, TLS, gateway policy, timeout và network partition cần lớp test/monitor khác.

Hành trình nhiều service

Một pact chỉ chứng minh một ranh giới; không chứng minh toàn bộ user journey.

Chất lượng API design

Hai phía có thể tương thích với một API vẫn khó dùng, thiếu nhất quán hoặc kém an toàn.

Kết hợp: **Unit** cho logic · **Contract** cho compatibility · **Integration/E2E** cho wiring và critical journeys.

Bắt đầu nhỏ, khóa đúng **một ranh giới rủi ro**

- 1 Chọn một Consumer–Provider có thay đổi thường xuyên hoặc hay gây tích hợp.
- 2 Contract 1–2 luồng quan trọng: success + một error behavior có giá trị.
- 3 Chạy consumer test và provider verification trong CI của từng đội.
- 4 Thêm Broker, version metadata và `can-i-deploy` khi workflow ổn định.
- 5 Đo: lỗi compatibility phát hiện trước staging, thời gian feedback, số lần deploy bị chặn đúng.

TAKEAWAY

Contract Testing bảo vệ sự hiểu nhau giữa các service

Fast

Feedback sớm tại local và CI.

Focused

Khoanh đúng ranh giới Consumer-Provider.

Safe

Cho phép các service tiến hóa độc lập có kiểm soát.

Q & A